Tabla de contenidos

1. Clases de Excepciones
2. Excepciones personalizadas
    a. Un ejemplo: La lista del curso

Clases de Excepciones

En Python, todas las excepciones heredan de `BaseException`. A partir de ella existen tres tipos de excepciones: **`SystemExit`**, **`KeyboardInterrupt`**, y **`Exception`**. Todas las excepciones generadas por errores durante la ejecución de un programa son subclases de `Exception`, tal como se muestra en el siguiente diagrama:

Esto quiere decir, que si se usa `Exception` para manejar errores (`except Exception`), esto capturará cualquier error que sea instancia de una subclase de `Exception`. De esta forma, el tratamiento es general y no específico a un tipo de error en especial. En general, es recomendable actuar de forma **selectiva** sobre un tipo determinado de excepciones (`IOError`, `AtributeError`, `ValueError`, etc.). Si bien es posible manejar cualquier tipo de excepción capturando `Exception`, esto se considera una **mala práctica**, pues capturar una excepción sin saber su naturaleza, puede causar comportamientos inesperados en el programa.

```
def divide(num, den):
    if not (isinstance(num, int) and isinstance(den, int)):
        raise TypeError("Type error in numerator or denominator. :'(")

    if num < 0 or den < 0:
        raise ValueError("There is a negative value in numerator or
denominator >:(")

    return float(num) / float(den)

try:
    print(divide(1, "1"))
except Exception as err:
    # Este bloque opera para todos los tipos de excepciones que
hereden de Exception.
    print(f"Error: {err}")
    print("Check the input data. Something went wrong, but I don't
know the specific type of Exception...")
```

```
Error: Type error in numerator or denominator. :'(
Check the input data. Something went wrong, but I don't know the
specific type of Exception...
```

```
try:
    print(divide(1, -2))
except Exception as err:
    # Este bloque opera para todos los tipos de excepciones que
hereden de Exception.
    print(f"Error: {err}")
    print("Check the input data. Something went wrong, but I don't
know the specific type of Exception...")
```

```
Error: There is a negative value in numerator or denominator >:(
Check the input data. Something went wrong, but I don't know the
specific type of Exception...
```

```
try:
    print(divide(4, 0))
except Exception as err:
    # Este bloque opera para todos los tipos de excepciones que
hereden de Exception.
    print(f"Error: {err}")
    print("Check the input data. Something went wrong, but I don't
know the specific type of Exception...")
```

```
Error: float division by zero
Check the input data. Something went wrong, but I don't know the
specific type of Exception...
```

Excepciones personalizadas

Podemos crear nuestros propios tipos de excepciones. Para ello debemos heredar desde la clase `Exception`. Podemos modificar el comportamiento heredado sobrescribiendo los métodos que tiene implementada esta clase.

```
class Exception1(Exception):

    # Al no sobrescribir nada, hereda todo sin modificaciones.
    pass


class Exception2(Exception):

    def __init__(self, a, b):
        # Sobrescribimos el __init__ para cambiar el ingreso de los
parámetros.
        super().__init__(f"One of the values {a} or {b} is not an
integer")


def divide(num, den):
```

```
    # Por ejemplo, redefiniremos las excepciones que
    # utilizamos en los ejemplos anteriores.
    if not (isinstance(num, int) and isinstance(den, int)):
        raise Exception2(num, den)

    if num < 0 or den < 0:
        raise Exception1("The values are negative")

    return float(num) / float(den)

# Este ejemplo lanza una Excepcion1.
try:
    print(divide(4, -3))

except Exception1 as err:
    # Este bloque opera para la Excepcion1.
    print(f"Error: {err}")

except Exception2 as err:
    # Este bloque opera para Excepcion2 cuando los datos no son
enteros.
    print(f"Error: {err}")
```

```
Error: The values are negative
```

```
# Este ejemplo lanza una Excepcion2.
try:
    print(divide(4.4, -3))

except Exception1 as err:
    # Este bloque opera para la Excepcion1.
    print(f"Error: {err}")

except Exception2 as err:
    # Este bloque opera para Excepcion2 cuando los datos no son
enteros.
    print(f"Error: {err}")
```

```
Error: One of the values 4.4 or -3 is not an integer
```

Podemos definir comportamientos personalizados para las excepciones que creamos. Por ejemplo, agregar métodos que nos permitan recuperar información de la excepción.

```
class TransactionError(Exception):

    def __init__(self, funds, expense):
        super().__init__(f"The money in the wallet is not enough to
pay ${expense}")
        self.funds = funds
        self.expense = expense
```

```
    def excess(self):
        return self.expense - self.funds


class Wallet:

    def __init__(self, money):
        self.funds = money

    def pay(self, expense):
        if self.funds - expense < 0:
            raise TransactionError(self.funds, expense)
        self.funds -= expense


w = Wallet(1000)

try:
    w.pay(1500)

except TransactionError as err:
    print(f"Error: {err}. There is an overspending of $
{err.excess()}.")
```

```
Error: The money in the wallet is not enough to pay $1500. There is an
overspending of $500.
```

Un ejemplo: La lista del curso

En este ejemplo desea modelar la siguiente situación: se desean inscribir estudiantes en distintos cursos universitarios, para lo cual se cuentan con listas de las peticiones de varios estudiantes por curso. El problema es que estas listas fueron afectadas por *hackers* que poblaron de información falsa, específicamente, hay estudiantes con número de identificación duplicados. Ya que no hay forma de identificar los falsos de los verdaderos, se inscribirán solamente quienes tengan un identificador nuevo, y los duplicados se almacenarán de alguna forma, para futura referencia.

La siguiente solución utiliza una excepción personalizada para llevar el registro de los duplicados encontrados en la lista. Aprovecha el uso de atributos de clase para mantener un registro general de todos los duplicados, y atributos de instancia para obtener información.

> **(Recordatorio...**
>
> Al usar clases, normalmente definimos atributos para las instancias mediante el uso de `self.attribute = valor`. Esto genera la creación de un **atributo de instancia**, cuyo valor es accesible desde cualquier punto de la instancia mediante `self`. Si definimos un atributo al nivel de los métodos, se le conoce como **atributo de clase** y es accesible por todas las instancias mediante el nombre de la clase: `Class.attribute`. También, es posible acceder mediante `self` en una instancia, siempre y cuando no tenga un atributo de instancia del mismo nombre definido.

Cuando se busca un valor de atributo mediante `self.attribute`, el orden de búsqueda es:

1. Buscar en la instancia. (atributo de instancia)
2. Si no existe, buscar en la clase. (atributo de clase)
3. Si no existe, levantar un error (`AttributeError`).

...cierra recordatorio.)

```
from collections import namedtuple, defaultdict


# Modelamos les estudiantes como entidades simples.
Student = namedtuple("Student", ["id_number", "first_name",
"last_name"])


class DuplicateStudentError(Exception):

    # Se mantiene un diccionario como atributo de clase para almacenar
todos los duplicados.
    duplicate_students = defaultdict(list)

    def __init__(self, student):
         # Se almacena al estudiante que lanzó la excepción.
        self.student = student
        id_number = student.id_number
        # Se almacena el duplicado.
        self.duplicate_students[id_number].append(student)
        super().__init__(f"Duplicate student error with ID number:
{student}")

    # Creamos property de instancia que accede a todos los duplicados
del mismo número.
    @property
    def duplicates(self):
        return self.duplicate_students[self.student.id_number]

     # Creamos property de instancia que calcula la cantidad de
duplicados del mismo número.
    @property
    def count(self):
        return len(self.duplicate_students[self.student.id_number])

# Modelamos el curso con una estructura simple para almacenar sus
estudiantes
class Course:

    def __init__(self, name):
        self.name = name
        self.students = dict()
```

```
    # Método encargado de inscribir nuevo estudiante, en caso de
encontrar duplicado, levanta un error.
    def enroll(self, student):
        if student.id_number in self.students:
            raise DuplicateStudentError(student)
        else:
            self.students[student.id_number] = student
            print(f" Student with ID {student.id_number} enrolled
successfully.")

students_to_enroll = [
    Student('15633459', 'Juan', 'Hernández'),
    Student('1663525J', 'Belén', 'Pinto'),
    Student('17632451', 'Ariel', 'Gonzalez'),
    Student('15633459', 'Fernanda', 'Errazuriz'),
    Student('1563001J', 'Javiera', 'Martínez'),
    Student('1663525J', 'Benjamín', 'Valdivieso'),
    Student('15633459', 'Cristian', 'Soto'),
]

course_iic2233 = Course('IIC2233')

for student in students_to_enroll:
    try:
        course_iic2233.enroll(student)
    except DuplicateStudentError as error:
        print(f" Duplicate student with ID {student.id_number}
found.")
        print(f"There are now {error.count} students with the
duplicate ID {student.id_number}:")
        print("\t" + ", ".join([f"{s.first_name} {s.last_name}" for s
in error.duplicates]))
```

```
 Student with ID 15633459 enrolled successfully.
 Student with ID 1663525J enrolled successfully.
 Student with ID 17632451 enrolled successfully.
 Duplicate student with ID 15633459 found.
There are now 1 students with the duplicate ID 15633459:
     Fernanda Errazuriz
 Student with ID 1563001J enrolled successfully.
 Duplicate student with ID 1663525J found.
There are now 1 students with the duplicate ID 1663525J:
     Benjamín Valdivieso
 Duplicate student with ID 15633459 found.
There are now 2 students with the duplicate ID 15633459:
     Fernanda Errazuriz, Cristian Soto
```